

# Leveraging Python Generators for Developing Elegant Code

Iterators in Python lead to a lot of overhead. Python generators are a simple way of creating iterators in which all overheads are automatically handled. The use of generators results in elegant and minimal code.



Figure 1: The Connect-4 game board

```
while num < n:
    yield num      ## generate next
                    number in the sequence
    num += 1

sum_of_first_n = sum(firstn(1000000))
```

In the above example, the entire list of the first one million numbers is not built in memory and then given to the sum function. Instead, the numbers in the sequence are generated on-the-fly. As the function *firstn* is called repeatedly, it remembers its previous state, specifically the value of *num* and continues execution from thereon. When execution reaches the *yield* statement, the value of *num* is returned and the state checkpointed. When the function is invoked again, it resumes from the statement after *yield* and continues. Eventually, when the function performs a regular return or falls off the end, the state is destroyed, and the generator terminates.

As the per information provided at <https://wiki.python.org/moin/Generators>, the power of generator expressions is derived from being “... very similar to the implementation that built a list in memory, but has the memory usage characteristic of the iterator implementation.”

However, programmers accustomed to a single ‘flow of control’ model, especially as supported in the C family of languages, may not find generators very intuitive,

The Python programming language has many constructs that enable programmers to write elegant and minimal code. Other than its aesthetic charm, elegant code has practical benefits, primarily with regard to readability and, thereby, maintainability. Such code typically comes out as cleanly separating the different aspects of the solution, which helps the maintainer both in fixing bugs and extending the code for adjacent requirements. As Python provides a wide set of features, some

of them are occasionally overlooked but each feature is effective for a particular type of problem. The generator construct is one such feature that allows one to write elegant code. The generator functionality is realised with the *yield* statement.

We start with looking at a simple example of how generators operate (reference [1]):

```
# a generator that yields items instead
of returning a list
def firstn(n):
    num = 0
```